LEI/LSIRC · Sistemas Distribuídos — Trabalho Prático

Leilões em tempo real

Unidade curricular	Sistemas Distribuídos (LEI + LSIRC), 2026/2027
Docente	Ricardo Costa — rcosta@estg.ipp.pt
Início	semana de 09/11/2026
Entrega	11/12/2026, no Moodle
Defesas	14 a 18/12/2026
Grupos	até 2 elementos

1. Descrição

Construir um **sistema distribuído de leilões em tempo real**. Um servidor guarda os leilões; vários clientes de consola, em processos diferentes, registam-se, criam leilões, licitam e acompanham o que se passa. Cada licitação aceite é difundida a quem acompanha o leilão e, no fecho, o vencedor é anunciado a todos.

Não se pede «uma aplicação com sockets». Pede-se que o sistema trate, com requisitos que se podem **verificar a funcionar**, cinco problemas de um sistema distribuído: **concorrência** (clientes a alterar o mesmo leilão ao mesmo tempo), **protocolo** (mensagens e sequências acordadas entre processos), **difusão** (um aviso, muitos destinatários, alguns avisos perdem-se), **tempo** (quem decide que o leilão fechou) e **falhas** (o servidor reinicia ou cai).

Exemplo de troca, na sintaxe do protocolo base fornecido (Anexo A); o aviso multicast é desenhado por vós:

```
ana> LOGIN ana segredo1
srv> OK
ana> CRIAR 100 5 300 Relógio de bolso     (base, incremento, duração em s)
srv> OK id=1
rui> LICITAR 1 105                        (o rui já fez LOGIN)
srv> OK n=1                               (aceite: 1.ª licitação do leilão 1)
rui> LICITAR 1 106
srv> ERRO BAIXA valor inferior ao minimo
multicast -> aviso: leilão 1, n.º 1, valor 105, rui   (formato vosso)
```

2. Definições (valem para todos os níveis)

- **Leilão**: identificador, artigo, preço base, incremento mínimo, criador e hora de fecho. Preços e licitações são **valores inteiros**; o incremento é positivo. O cliente indica a **duração**; a hora de fecho é calculada pelo **relógio do servidor**, o único usado para decidir: o leilão está **aberto se e só se a hora do servidor é anterior à hora de fecho** (no Completo, a hora de fecho pode ser adiada, C2, e o leilão pode ser cancelado, C1).
- **Licitação válida**: pedida por um utilizador autenticado que **não** é o criador, num leilão que existe e está aberto, com valor não inferior ao preço base (se ainda não há licitação aceite) ou ao melhor valor mais o incremento (caso contrário). O vencedor atual pode voltar a licitar.
- **Aceite**: licitação válida no instante em que o servidor a processa. Recebe um número de ordem no leilão (1, 2, 3, …) e fica **gravada no histórico antes de o servidor responder OK**. Uma licitação inválida é **rejeitada**, com resposta de erro e motivo, e não altera nada.
- **Concorrentes**: duas licitações ao mesmo leilão que chegam ao servidor ao mesmo tempo. O resultado tem de ser o de as processar uma a seguir à outra, por alguma ordem: nenhuma se perde, nenhuma se sobrepõe, e a segunda é avaliada contra o estado que a primeira deixou.
- **Vencedor**: autor da última licitação aceite; sem licitações, o leilão fecha sem vencedor. Um leilão cancelado também não tem vencedor.
- **Aviso**: datagrama multicast cujo formato é desenhado por vós, com pelo menos o leilão e o **número de sequência por leilão** (1, 2, 3, … um por licitação aceite, fecho ou cancelamento). Há **perda** quando um cliente que acompanha um leilão recebe um aviso com número superior ao último visto mais um.

- **Cancelado** (nível Completo): leilão aberto que a consola de administração fecha antes da hora, **sem vencedor**. Conta como fecho; no Anexo A está `FECHADO` com `vencedor=-`.

3. Níveis e requisitos

Cada nível inclui os anteriores e indica a nota a que se pode aspirar: **Base** até cerca de 12, **Intermédio** até cerca de 16, **Completo** até 20. Um requisito conta quando o docente o **vê a funcionar**.

Nível Base

- **B1 Servidor TCP que atende vários clientes ao mesmo tempo** (uma *thread* por cliente ou *pool*). porto configurável, 5050 por omissão. Uma *pool* tem de ter *threads* para todos os clientes ligados ao mesmo tempo (por exemplo, `newCachedThreadPool`): com uma *pool* fixa mais pequena, um teste com muitos clientes ao mesmo tempo bloqueia.
- **B2 Protocolo do Anexo A cumprido à letra**: um cliente de outro grupo fala com o vosso servidor. As **extensões** que desenharem (aviso multicast, pedido de estado, comandos novos e, no Completo, a interface RMI) são **documentadas no relatório**; nunca alteram os comandos do anexo. Mensagem mal formada ou fora de sequência recebe o `ERRO` do anexo; o servidor não cai. O verificador de conformidade (`Conformidade.java`, no Moodle, Anexo A.6) é **público**: usem-no antes da entrega; o docente usa o mesmo programa para G2.
- **B3** Registo e *login* (`REGISTAR` e `LOGIN` do Anexo A). A palavra-passe **nunca** aparece em claro (nem em ficheiros, nem no histórico, nem em logs): guarda-se só o resumo SHA-256 (`MessageDigest`). Sem *login* só se pode registar, listar, consultar e sair (ver o anexo).
- **B4 Criar, listar, consultar e licitar** leilões, com os comandos `CRIAR`, `LISTAR`, `CONSULTAR` e `LICITAR` do Anexo A. o formato exato das respostas (B2).
- **B5 Licitação atómica** (secção 2): licitações simultâneas no mesmo leilão nunca se perdem nem se sobrepõem. O docente verifica-o com um teste de concorrência próprio, que usa só os comandos do Anexo A.
- **B6** Licitação a um leilão fechado (hora do servidor igual ou posterior à de fecho) é rejeitada.
- **B7 Histórico persistente** em ficheiro, escrito pelo servidor: registos (com o resumo da palavra-passe, B3), leilões criados e licitações aceites (no Intermédio, também os fechos). Cada licitação aceite é gravada **antes** de o servidor responder OK. Sem base de dados: o formato do ficheiro (texto ou binário) é livre, decidido pelo grupo e descrito no relatório. Ao arrancar, o servidor lê o ficheiro: os utilizadores, os leilões e os melhores valores ficam repostos. Uma última linha truncada (o servidor morreu a gravar) não impede o arranque.

Nível Intermédio

- **I1 Cada licitação aceite gera um aviso multicast UDP com número de sequência por leilão.** o fecho também gera aviso; grupo e porto configuráveis (por omissão 239.255.0.1 e 6060, um grupo único para todos os leilões, com o identificador do leilão no aviso).
- **I2** O cliente acompanha um leilão e mostra os avisos em tempo real; **ao detetar perda, recupera o estado por TCP**. o pedido de estado é vosso (podem usar `CONSULTAR`, cujo `licitacoes` dá o número de sequência das licitações; o do fecho e o do cancelamento tratam à parte ou num comando vosso); o estado inclui o número de sequência atual e o cliente continua a partir dele, mostrando o estado atual e os avisos seguintes, todos e uma só vez (não se exige mostrar as licitações perdidas); o cliente tem uma opção que descarta 1 de cada N avisos recebidos, para simular perdas na demonstração.
- **I3 À hora de fecho, sem pedido de ninguém, o servidor fecha o leilão e anuncia o vencedor** (ou «sem vencedor»). grava o fecho no histórico; atraso até 1 s; uma licitação nesse intervalo é rejeitada (B6).
- **I4** Se o servidor esteve parado à hora de fecho, ao arrancar fecha e anuncia os leilões vencidos.

Nível Completo (Java RMI, PL05)

- **C1 Consola de administração, num processo separado e por Java RMI: lista os leilões e os utilizadores com sessão iniciada e cancela um leilão aberto.** o registo RMI é criado no próprio servidor, num porto diferente do TCP (por exemplo 5051); o cancelamento fecha o leilão **sem vencedor**, fica gravado no histórico (repõe-se no arranque) e gera um aviso multicast (conta como fecho na sequência do leilão); depois de a consola receber a confirmação, nenhuma licitação é aceite nesse leilão e uma licitação que já recebeu OK não se perde. As *threads* TCP e as *threads*

RMI alteram **a mesma estrutura de estado**: cancelar ao mesmo tempo que alguém licita tem de continuar atómico (conta em G1). A interface remota é livre (consola e servidor são do mesmo grupo): o relatório lista as operações.

- **C2 Anti-*sniping*: uma licitação aceite nos últimos N segundos (N configurável, por exemplo 30) adia o fecho para agora + N, pelo relógio do servidor.** uma licitação fora desse intervalo não adia nada e o fecho nunca é antecipado; o prolongamento é anunciado por multicast (por exemplo no aviso da própria licitação, sem outro número de sequência: documentem a vossa escolha); o fecho automático (I3) usa a hora nova; `CONSULTAR` mostra o `restante` atualizado; a hora nova fica no histórico e repõe-se no arranque.
- *Pistas*: a consola não guarda estado, as *threads* RMI chamam os mesmos métodos protegidos que as TCP; guardem uma referência ao objeto remoto (senão o *garbage collector* recolhe-o e a consola recebe `NoSuchObjectException`); `-Djava.rmi.server.hostname=localhost` evita um *stub* que aponta para um endereço que a consola não alcança.

4. Requisitos técnicos

- Java 17 ou superior, **só biblioteca padrão**, interface de consola (interface gráfica não é valorizada). Compila e corre pela linha de comandos: `javac --release 17 -encoding UTF-8` e `java`. Quem usa um IDE entrega também o código que compila sem ele.
- Portos a partir de 1024; **nunca 5000** (ocupado pelo AirPlay no macOS). Multicast: correr com `-Djava.net.preferIPv4Stack=true`. A rede do laboratório pode bloquear multicast entre máquinas: a demonstração em `localhost` é aceite. Cada grupo configura o seu porto UDP para não receber os avisos de outros.
- O servidor não decide nada com o relógio de um cliente. O estado partilhado nunca é lido nem alterado sem proteção.

5. O que entregar

Um ficheiro zip no Moodle, com o número de aluno como nome (grupo de 1) ou os dois números separados por hífen (grupo de 2), por exemplo `8230001.zip` ou `8230001-8230002.zip`, com:

1. **Código-fonte** e **README** com os comandos para compilar e correr servidor, consola e clientes, do zero (compilar e arrancar em poucos minutos), e com o **nível que o grupo declara**. Um **comando pronto por cenário** que o nível exige (reinício, aviso perdido, fecho, cancelamento e anti-*sniping*), para o docente os ver a correr depressa.
2. **Relatório** em PDF, com o máximo de 6 páginas: extensões ao protocolo do Anexo A (formato do aviso multicast, pedido de estado e, no Completo, as operações RMI: mensagens, erros, sequência); arquitetura (processos, *threads*, estado partilhado); **formato do ficheiro de histórico** e como é lido no arranque; decisões de **concorrência** (ficheiro e método da região crítica, e porquê), de **consistência** (o que significa «aceite», e quem ganha quando uma licitação e um cancelamento chegam ao mesmo tempo) e de **falhas** (o que acontece a cada falha: cliente, servidor, aviso perdido); **limitações** assumidas; requisitos cumpridos; e a **declaração do uso de IA**: onde e para quê.

6. Avaliação

A avaliação contínua é a nota deste trabalho (0 a 20). A nota resulta dos 10 critérios abaixo, de 0 a 2 valores cada, tendo em conta o nível atingido (secção 3). A defesa é obrigatória; o G10 é avaliado para cada elemento.

Critério	Nível	O que se verifica
G1 Concorrência e atomicidade	Base	teste do docente passa; região crítica e estado partilhado protegidos (no Completo, também perante o cancelamento)
G2 Protocolo	Base	Anexo A cumprido à letra; extensões documentadas; mensagem inválida não derruba
G3 Registo e autenticação	Base	palavra-passe guardada como resumo SHA-256; operações exigem sessão
G4 Histórico e recuperação	Base	parar e reiniciar o servidor repõe o estado
G5 Difusão e perdas	Intermédio	multicast com sequência; deteção e recuperação

Critério	Nível	O que se verifica
G6 Tempo e fecho	Base/Intermédio	recusa tardia; fecho automático pelo relógio do servidor; vencedor anunciado
G7 Administração e anti-*sniping*	Completo	consola RMI (listar, utilizadores com sessão iniciada, cancelar); prolongamento do fecho
G8 Qualidade do código	todos	organização e legibilidade; erros tratados com mensagens claras; sockets, ficheiros e *threads* fechados
G9 Relatório e README	todos	reproduz-se do zero; decisões e limitações justificadas
G10 Defesa	todos, por elemento	explica o percurso de uma licitação e responde a «e se…»

7. Grupos e fontes

- **Grupos** de 1 ou 2 elementos [[A CONFIRMAR: data limite de constituição de grupos]].
- **Plágio, fontes e IA.** Trabalho copiado, mesmo em parte, é excluído da avaliação. Citem todas as fontes (código, documentação, colegas). O uso de ferramentas de IA é **permitido e tem de ser declarado** no relatório (onde e para quê). Na defesa, cada elemento tem de saber explicar o código entregue, venha de onde vier (conta no G10).

Anexo A — Protocolo base

Este protocolo é **fornecido** e tem de ser cumprido à letra: um cliente de um grupo tem de conseguir falar com o servidor de outro. Tudo o resto (aviso multicast, pedido de estado, interface RMI da consola) é desenhado por vós e documentado no relatório. Podem **acrescentar** comandos, nunca alterar os que aqui estão.

A.1 Formato. TCP, porto 5050 por omissão. Texto UTF-8, **uma mensagem por linha**, terminada por `\n` (o servidor aceita também `\r\n` e envia sempre `\n`). Campos separados por **um** espaço; sem espaço no início nem no fim da linha e sem campos vazios; espaços a mais ou no fim são `SINTAXE`. **Exceção, `CRIAR`:** tudo o que vem depois do 4.º espaço é o artigo; um artigo vazio, ou que começa ou acaba em espaço, é `VALOR`; espaços dentro do artigo, mesmo seguidos, fazem parte dele. Comandos em maiúsculas. Uma linha com mais de 1024 *caracteres* (não *bytes*) é um erro. Cada pedido recebe **uma** resposta, pela ordem dos pedidos; o servidor não envia nada por iniciativa própria (os avisos vão por multicast). Uma resposta de erro nunca fecha a ligação.

A.2 Campos.

- **utilizador**: 3 a 20 caracteres entre `A-Z`, `a-z`, `0-9` e `_`. **palavra-passe**: 1 a 64 caracteres, sem espaços nem caracteres de controlo.
- **números**: inteiros decimais sem sinal, 1 a 9 dígitos (zeros à esquerda aceites). **id** e **incremento**: de 1 a 999 999 999; **base** e **valor**: de 0 a 999 999 999; **duração**: segundos, de 1 a 86 400. Fora destes intervalos é `VALOR`.
- **artigo**: texto livre de 1 a 60 *caracteres*, o **último campo** de `CRIAR`: ocupa o resto da linha, pode ter espaços e acentos, não começa nem acaba em espaço e não tem caracteres de controlo. *Decisão:* só o artigo pode ter espaços e vai no fim, por isso não há aspas nem *escapes*; os outros campos nunca têm espaços e uma linha divide-se com um só `split`.

A.3 Respostas. `OK` ou `OK <dados>`, ou `ERRO <CODIGO> <motivo>`. O motivo é texto livre para humanos (não vazio, nunca interpretado): os programas usam só o código. Todas as respostas têm uma linha, exceto `LISTAR`.

Código	Quando
`SINTAXE`	comando desconhecido, número de campos errado, espaços a mais (exceto no artigo de `CRIAR`), linha vazia ou com mais de 1024 caracteres
`VALOR`	campo bem formado mas fora das regras de A.2 (não numérico, fora do intervalo, utilizador ou artigo inválido)
`SEM_SESSAO`	o comando exige `LOGIN` e a ligação não o fez
`JA_AUTENTICADO`	`LOGIN` numa ligação que já tem sessão
`CREDENCIAIS`	utilizador inexistente ou palavra-passe errada (mesma resposta para os dois)
`EXISTE`	`REGISTAR` de um utilizador que já existe
`INEXISTENTE`	o leilão não existe
`FECHADO`	o leilão já fechou (hora do servidor igual ou posterior à de fecho) ou foi cancelado (C1)
`PROPRIO`	o criador tenta licitar no seu leilão
`BAIXA`	valor inferior ao preço base (sem licitações) ou ao melhor valor mais o incremento
`INTERNO`	falha do servidor

Ordem de verificação (responde-se com o primeiro erro que se aplica): em `CRIAR` e `LICITAR`, `SINTAXE`, `SEM_SESSAO`, `VALOR`, `INEXISTENTE`, `FECHADO`, `PROPRIO`, `BAIXA`; em `LOGIN`, `SINTAXE`, `JA_AUTENTICADO`, `CREDENCIAIS`; em `REGISTAR`, `SINTAXE`, `VALOR`, `EXISTE`; nos outros, `SINTAXE`, `VALOR`, `INEXISTENTE`.

A.4 Comandos.

Comando	Sessão	Resposta OK	Erros possíveis
`REGISTAR <utilizador> <palavra-passe>`	não	`OK` (não inicia sessão)	`SINTAXE` `VALOR` `EXISTE`
`LOGIN <utilizador> <palavra-passe>`	só sem sessão	`OK`; a sessão dura até `SAIR` ou ao fim da ligação	`SINTAXE` `JA_AUTENTICADO` `CREDENCIAIS`

Comando	Sessão	Resposta OK	Erros possíveis
`CRIAR <base> <incremento> <duração> <artigo>`	exige	`OK id=<id>` (inteiro positivo, escolhido pelo servidor)	`SINTAXE SEM_SESSAO VALOR`
`LISTAR`	não	`OK <n>` e depois `n` linhas `LEILAO <registo>`, por id crescente	`SINTAXE`
`CONSULTAR <id>`	não	`OK <registo>`	`SINTAXE VALOR INEXISTENTE`
`LICITAR <id> <valor>`	exige	`OK n=<k>`: k é o número de ordem da licitação no leilão (1, 2, …)	`SINTAXE SEM_SESSAO VALOR INEXISTENTE FECHADO PROPRIO BAIXA`
`SAIR`	não	`OK`, e o servidor fecha a ligação; o que vier depois de `SAIR` nessa ligação é ignorado	`SINTAXE`

Registo de leilão, campos `nome=valor` por esta ordem, lidos pelo nome (o `artigo` é o resto da linha): `id` `estado` (`ABERTO` ou `FECHADO`) `melhor` (0 se não há licitações) `vencedor` (`-` se não há) `criador` `base` `incremento` `licitacoes` (número de licitações aceites) `restante` (segundos até ao fecho, arredondado para cima, por isso um leilão de 60 s consultado menos de 1 s depois de `CRIAR` tem `restante=60`; 0 se fechado) `artigo`. Um leilão **cancelado** (C1) está `FECHADO`, com `vencedor=-` e `restante=0`; os outros campos mantêm-se. Com anti-*sniping* (C2), `restante` acompanha a hora de fecho atual.

A.5 Exemplo (duas ligações, `c1` e `c2`; o servidor responde a cada linha):

```
c1> REGISTAR ana segredo1                  OK
c1> LOGIN ana segredo1                     OK
c1> CRIAR 100 5 300 Relógio de bolso       OK id=1
c1> LICITAR 1 100                          ERRO PROPRIO o criador nao pode licitar o seu leilao
c2> REGISTAR rui segredo2                  OK
c2> LICITAR 1 100                          ERRO SEM_SESSAO faca LOGIN primeiro
c2> LOGIN rui segredo2                     OK
c2> LICITAR 1 100                          OK n=1
c2> LICITAR 1 102                          ERRO BAIXA valor inferior ao minimo
c2> LICITAR 1 abc                          ERRO VALOR id ou valor invalidos
c2> LICITAR 9 100                          ERRO INEXISTENTE leilao nao existe
c2> CONSULTAR 1                            OK id=1 estado=ABERTO melhor=100 vencedor=rui
      criador=ana base=100 incremento=5 licitacoes=1 restante=287 artigo=Relógio de bolso
c2> LISTAR                                 OK 1
                                           LEILAO id=1 estado=ABERTO melhor=100 ... (a mesma linha)
c2> listar                                 ERRO SINTAXE comando desconhecido
c2> SAIR                                   OK        (o servidor fecha a ligação)
```

(Cada resposta de `CONSULTAR` e de `LEILAO` cabe numa só linha; aqui foi quebrada para caber na página.)

A.6 Verificador de conformidade. No Moodle está `Conformidade.java` (material do trabalho). Com o vosso servidor a correr: `javac --release 17 -encoding UTF-8 Conformidade.java` e `java Conformidade localhost 5050` (host e porto opcionais). Executa 30 casos do anexo (9 do caminho normal, [N], e 21 de erros e casos de fronteira, [E]), e escreve `PASSA` ou `FALHA` por caso, com o que esperava e o que recebeu; no fim, um resumo. Se o servidor nunca responder, termina ao fim de 5 s com uma mensagem. Cria utilizadores e leilões novos em cada execução, demora cerca de 4 s e não depende do anti-*sniping* (serve com ele ligado). Não testa concorrência, persistência, multicast, RMI nem as vossas extensões. O docente usa o mesmo programa para pontuar o G2.